

Intermediate (.env by hand) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Where this exercise is saved: `practice/env_config_practice.py`, under its `# Intermediate` section — the same file also holds the [Edge cases exercise](#), under an `# Edge cases` section. Run it with `cd practice && python env_config_practice.py`.

Used later by: the [Build Task](#)'s `load_config()` in `practice/build_task/config.py` — it ships the short `load_dotenv() + os.getenv()` version you land on here, re-written (not imported) so it can return a typed `Config` object.

All examples below assume a `.env` file next to your script — `practice/.env` — containing:

```
# practice/.env
# credentials for the API client
OPENAI_API_KEY="sk-test123"
LOG_LEVEL=DEBUG
```

Basic Version

Approach 1 — the direct way

```
# practice/env_config_practice.py – Intermediate section
def parse_env_by_hand(path):
    env_vars = {}
    for line in open(path):
        line = line.strip()
        if line == "" or "=" not in line:
            continue
        parts = line.split("=", 1)
        env_vars[parts[0]] = parts[1]
    return env_vars

values = parse_env_by_hand(".env")
print(values["OPENAI_API_KEY"])
print(values["LOG_LEVEL"])
```

Expected output:

```
"sk-test123"  
DEBUG
```

Notice `OPENAI_API_KEY` still has its quote marks in the output — this version doesn't know to remove them, it just splits on `=` and keeps whatever's on each side. The comment line (`# credentials...`) is skipped safely too, but only because it happens to contain no `=`, not because this version recognizes it as a comment on purpose.

This version works correctly for what the exercise asks. It's missing type hints, and it builds the dictionary with `parts[0]` / `parts[1]` instead of unpacking directly — both fine for a first working version.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — type hints and tuple unpacking

```
# practice/env_config_practice.py – Intermediate section
def parse_env_by_hand(path: str) -> dict[str, str]:
    env_vars: dict[str, str] = {}
    for line in open(path):
        line = line.strip()
        if not line or "=" not in line:
            continue
        key, value = line.split("=", 1)
        env_vars[key] = value
    return env_vars

values = parse_env_by_hand(".env")
print(values["OPENAI_API_KEY"])
print(values["LOG_LEVEL"])
```

Expected output:

```
"sk-test123"
DEBUG
```

Approach 2 — a docstring, and using `with` to open the file properly

```
# practice/env_config_practice.py – Intermediate section
def parse_env_by_hand(path: str) -> dict[str, str]:
    """Read a simple KEY=value file and return it as a dict."""
    env_vars: dict[str, str] = {}
    with open(path) as env_file:
        for line in env_file:
            line = line.strip()
            if not line or "=" not in line:
                continue
            key, value = line.split("=", 1)
            env_vars[key] = value
```

```

return env_vars

values = parse_env_by_hand(".env")
print(values["OPENAI_API_KEY"])
print(values["LOG_LEVEL"])

```

Expected output:

```

"sk-test123"
DEBUG

```

Difference from Basic: both Intermediate approaches add full type hints (`path: str` , `-> dict[str, str]`) and unpack `key, value = line.split("=", 1)` directly instead of indexing into `parts[0]` / `parts[1]` . Approach 2 also switches to `with open(path) as env_file:` , which closes the file explicitly and correctly even if something inside the loop raises an exception — `for line in open(path):` (Approach 1, and the Basic Version) relies on the file being closed automatically once the loop finishes, which works but is less explicit.

In real code, `python-dotenv` (`load_dotenv()` then `os.getenv(...)` , 2 lines) already does this job for you, including handling comments and quoted values for free — the point of writing the parser above by hand wasn't to replace it, it was to understand what it's doing under the hood, so it stops feeling like a black box and starts feeling like "oh, that's just a loop with a dictionary." This is what the Build Task's `load_config()` actually ships.

Which one should you actually write? In real code, `python-dotenv` — always, for the parsing itself. The Intermediate version here is what you'd write if you genuinely couldn't use any library (rare) — but understanding it is what makes `python-dotenv` make sense once you get there, instead of feeling like a black box.